

SAP Ariba 连接 OpenMetadata：技术集成完整方案

关键结论

OpenMetadata 目前没有原生 SAP Ariba 连接器。但 SAP Ariba 提供多种数据暴露方式, 结合 OpenMetadata 的灵活架构, 有 4 种可行的技术方案, 企业可根据现有架构选择最适合的路径。

方案	复杂度	数据时效性	适合场景
方案一: Ariba → HANA/DB → OpenMetadata	低	准实时 (小时级)	已有 SAP HANA 或数据库集成
方案二: Ariba API → 自定义 Python 脚本 → OpenMetadata API	中	准实时 (小时级)	有开发能力, 需灵活定制
方案三: OpenMetadata Custom Connector	高	准实时	长期规划, 需持续维护
方案四: Ariba CSV 导出 → S3 → OpenMetadata	低	批量 (日级)	快速验证、数据量小

背景：SAP Ariba 的数据访问方式

SAP Ariba 是一个云原生 SaaS 平台, 不同于传统的 SAP ERP (有数据库可直连), Ariba 的数据主要通过以下 API 暴露：

Ariba 提供的 API 类型

API 类型	用途	数据量限制	制造业相关数据
Analytical Reporting API	报表数据导出	异步: 50,000 条/作业	采购订单、供应商绩效、物料分类
REST API (Procurement / Sourcing)	业务数据操作	同步: 50 条/页	供应商主数据、采购合同、物料目录
Supplier Data API	供应商数据	分页: 500 条/页	供应商注册信息、资质、评级
SOAP Web Service API	系统集成	无明确限制	供应商数据导入 (单向)
OData API	标准化查询	视配置而定	通过 BTP 中间层暴露

Ariba 认证方式

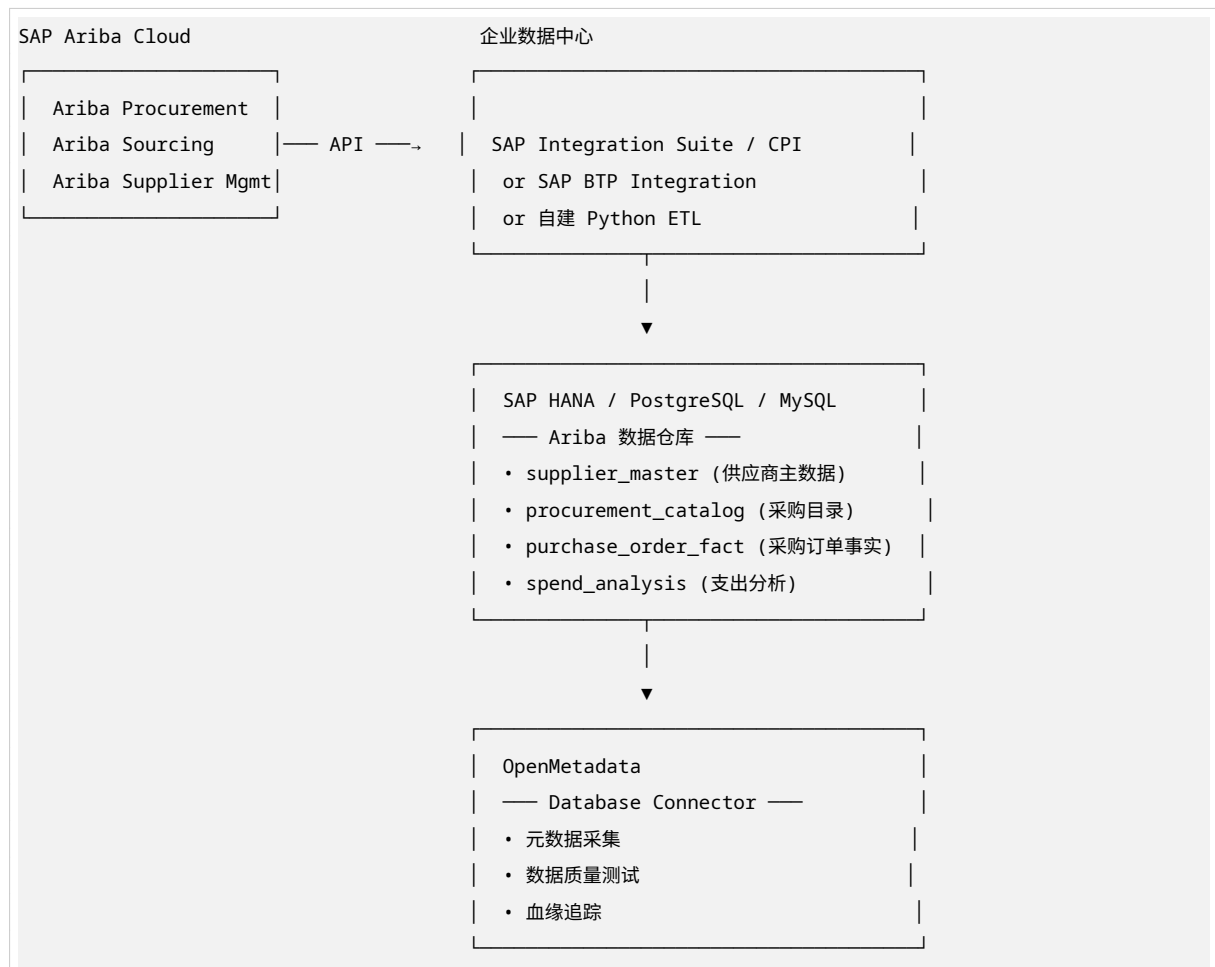
所有 Ariba API 都使用 **OAuth 2.0** 认证：

- **API Key**：在 Ariba Developer Portal 注册应用获取
- **OAuth Client ID / Secret**：用于获取 Access Token
- **Realm**：企业的 Ariba 租户标识（如 company-prod）

方案一：Ariba → SAP HANA/数据库 → OpenMetadata（推荐）

这是**最稳定、最可维护**的方案。利用 SAP 官方集成工具将 Ariba 数据同步到数据库，再用 OpenMetadata 的数据库连接器采集。

架构图



实施步骤

Step 1: Ariba 端准备（Ariba 管理员执行）

1. 登录 Ariba Developer Portal (<https://developer.ariba.com>)
2. 创建 Application:
 - Name: "OpenMetadata_Integration"
 - Type: Analytical Reporting API + Supplier Data API

3. 提交审批 (通常 1-3 个工作日)
4. 审批通过后获取:
 - API Key: xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
 - OAuth Client ID: openmetadata-client
 - OAuth Secret: xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
 - Realm: company-prod
5. 记录 Analytical Realm (报表数据在 Parent Realm 中)

Step 2: 建立数据同步通道 (3 种选择)

选择 A: SAP Integration Suite (CPI) — 最官方

```
# SAP 提供预构建的 iFlow 模板
# 参考 SAP Discovery Center Mission: "SAP Ariba Spend Analytics"

# 需要的组件:
# - SAP Integration Suite (CPI)
# - SAP HANA Cloud / SAP Datasphere (数据仓库)
# - OpenMetadata HANA Connector

# CPI 配置要点:
# 1. 创建 Security Material:
#   - OAuth2 Credentials (Ariba API 认证)
#   - JDBC Material (HANA 数据库连接)
# 2. 导入预构建 iFlow
# 3. 配置参数:
#   - Realm: company-prod
#   - API Key: xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
#   - Date Range: 动态参数 (最近 30 天)
# 4. 部署并调度: 每日凌晨 3 点执行
```

选择 B: 自建 Python ETL (最灵活)

```
#!/usr/bin/env python3
"""
SAP Ariba → PostgreSQL ETL Pipeline
使用 Analytical Reporting API (异步模式)
"""

import requests
import json
import time
import pandas as pd
from sqlalchemy import create_engine
from datetime import datetime, timedelta

# ===== 配置 =====
ARIBA_CONFIG = {
    "base_url": "https://openapi.ariba.com",
    "api_key": "YOUR_API_KEY",
    "oauth_url": "https://api.ariba.com/v2/oauth/token",
    "client_id": "YOUR_CLIENT_ID",
```

```

    "client_secret": "YOUR_CLIENT_SECRET",
    "realm": "company-prod"
}

DB_CONFIG = {
    "host": "postgres.company.internal",
    "port": 5432,
    "database": "ariba_dw",
    "user": "etl_user",
    "password": "YOUR_PASSWORD"
}

# ===== 1. 获取 OAuth Token =====
def get_access_token():
    """ 获取 Ariba OAuth Access Token """
    response = requests.post(
        ARIBA_CONFIG["oauth_url"],
        headers={"Content-Type": "application/x-www-form-urlencoded"},
        data={
            "grant_type": "client_credentials",
            "client_id": ARIBA_CONFIG["client_id"],
            "client_secret": ARIBA_CONFIG["client_secret"]
        }
    )
    response.raise_for_status()
    return response.json()["access_token"]

# ===== 2. 提交异步导出任务 =====
def submit_async_job(token, view_template, filters=None):
    """
    提交异步报表导出任务

    view_template: 报表视图模板名称
        - 'SupplierDimension' (供应商维度)
        - 'SupplierSpentFact' (供应商支出事实)
        - 'ItemMasterFact' (物料主数据事实)
        - 'PurchaseOrderFact' (采购订单事实)
    """
    url = f"{ARIBA_CONFIG['base_url']}/api/reporting-job/v1/{ARIBA_CONFIG['realm']}/jobs"

    payload = {
        "viewTemplateName": view_template,
        "format": "csv",          # CSV 格式
        "locale": "zh_CN",       # 中文本地化
        "filters": filters or {}
    }

    response = requests.post(
        url,
        headers={
            "Authorization": f"Bearer {token}",

```

```

        "x-api-key": ARIBA_CONFIG["api_key"],
        "Content-Type": "application/json"
    },
    json=payload
)
response.raise_for_status()
return response.json()["jobId"]    # 返回任务 ID

# ===== 3. 轮询任务状态 =====
def poll_job_status(token, job_id, max_retries=60):
    """ 轮询异步任务状态, 直到完成 """
    url = f"{ARIBA_CONFIG['base_url']}/api/reporting-job/v1/{ARIBA_CONFIG['realm']}/jobs/{job_id}"

    for i in range(max_retries):
        response = requests.get(
            url,
            headers={
                "Authorization": f"Bearer {token}",
                "x-api-key": ARIBA_CONFIG["api_key"]
            }
        )
        status = response.json()["status"]

        if status == "Completed":
            return response.json()["fileUrls"]    # 返回下载链接
        elif status == "Failed":
            raise Exception(f"Job failed: {response.json().get('errorMessage')}")

        print(f"Job status: {status}, retrying... ({i+1}/{max_retries})")
        time.sleep(30)    # 每 30 秒检查一次

    raise TimeoutError("Job polling timed out")

# ===== 4. 下载并解析数据 =====
def download_and_parse(file_urls):
    """ 下载 CSV 文件并解析为 DataFrame """
    dfs = []
    for url in file_urls:
        df = pd.read_csv(url, compression='zip')
        dfs.append(df)
    return pd.concat(dfs, ignore_index=True)

# ===== 5. 加载到 PostgreSQL =====
def load_to_postgres(df, table_name, schema="ariba"):
    """ 将 DataFrame 加载到 PostgreSQL """
    engine = create_engine(
        f"postgresql://{DB_CONFIG['user']}:{DB_CONFIG['password']}@{DB_CONFIG['host']}:{DB_CONFIG['port']}/{DB_CONFIG['database']}"
    )

    # 使用 upsert (INSERT ON CONFLICT UPDATE)

```

```

df.to_sql(
    table_name,
    engine,
    schema=schema,
    if_exists='append',    # 或 'replace' 全量刷新
    index=False,
    method='multi',
    chunksize=1000
)

print(f"Loaded {len(df)} rows into {schema}.{table_name}")

# ===== 主流程 =====
def main():
    """ 每日 ETL 主流程 """
    print(f"ETL started at {datetime.now()}")

    # 1. 认证
    token = get_access_token()
    print(f"${vcenter{{\hbox{{\includegraphics[height=\ht\strutbox]{{/usr/share/docminer/emoji/2713.png}}}}} ${OAuth token obtained}")

    # 2. 定义要同步的报表
    reports = [
        {
            "view_template": "SupplierDimension",
            "table": "supplier_master",
            "filters": {"LastModifiedDate": (datetime.now() - timedelta(days=1)).strftime("%Y-%m-%d")}
        },
        {
            "view_template": "ItemMasterFact",
            "table": "procurement_catalog",
            "filters": {}
        },
        {
            "view_template": "PurchaseOrderFact",
            "table": "purchase_order_fact",
            "filters": {"CreatedDate": (datetime.now() - timedelta(days=7)).strftime("%Y-%m-%d")}
        }
    ]

    # 3. 逐个同步
    for report in reports:
        try:
            print(f"\nSyncing {report['view_template']}...")
            job_id = submit_async_job(token, report["view_template"], report["filters"])
            file_urls = poll_job_status(token, job_id)
            df = download_and_parse(file_urls)
            load_to_postgres(df, report["table"])
            print(f"${vcenter{{\hbox{{\includegraphics[height=\ht\strutbox]{{/usr/share/docminer/emoji/2713.png}}}}} ${report['table']}: {len(df)} rows synced")

```

```

except Exception as e:
    print(f"${\vcenter{{\hbox{{\includegraphics[height=\ht\strutbox]{{/usr/share/docminer/
    ↳ emoji/2717.png}}}}}}}${report['table']}: {e}")

print(f"\nETL completed at {datetime.now()}")

if __name__ == "__main__":
    main()

```

选择 C: SAP BTP + Cloud Integration (中间层)

```

# 适用于已有 SAP BTP 订阅的企业
# BTP 应用作为 Ariba 和 OpenMetadata 之间的中间层

# 架构:
# Ariba → BTP Cloud Integration → BTP HANA Cloud → OpenMetadata HANA Connector

# BTP 端配置:
# 1. 在 BTP Cockpit 中创建 Cloud Integration 实例
# 2. 创建 Destination (指向 Ariba API)
# 3. 创建 HANA Cloud 数据库实例
# 4. 配置 iFlow 定时同步数据

```

Step 3: 在 OpenMetadata 中配置数据库连接器

Ariba 数据同步到数据库后，按照标准数据库连接器配置：

```

// OpenMetadata PostgreSQL 连接器配置 (Ariba 数据仓库)
{
  "serviceName": "Ariba-DataWarehouse",
  "serviceType": "Postgres",
  "connection": {
    "config": {
      "type": "Postgres",
      "username": "openmetadata",
      "password": "${ARIBA_DW_PASSWORD}",
      "hostPort": "postgres.company.internal:5432",
      "database": "ariba_dw",
      "tableFilterPattern": {
        "includes": [
          "supplier_master",
          "procurement_catalog",
          "purchase_order_fact",
          "spend_analysis",
          "supplier_performance"
        ]
      }
    }
  }
}

```

Step 4: 配置数据质量测试 (制造业场景)

```

-- Test Library: Ariba 供应商数据质量测试

-- 1. 供应商编码非空
SELECT supplier_id, supplier_name
FROM {{ table_name }}
WHERE supplier_id IS NULL OR supplier_id = '';

-- 2. 供应商名称重复检测（一供多名）
SELECT supplier_name, COUNT(DISTINCT supplier_id) as id_count
FROM {{ table_name }}
GROUP BY supplier_name
HAVING COUNT(DISTINCT supplier_id) > 1;

-- 3. 供应商状态有效性
SELECT supplier_id, status
FROM {{ table_name }}
WHERE status NOT IN ('Active', 'Inactive', 'Blocked', 'UnderReview');

-- 4. 采购目录物料编码与 SAP 物料编码映射完整性
SELECT c.catalog_item_id, c.sap_material_code
FROM procurement_catalog c
LEFT JOIN sap_mara m ON c.sap_material_code = m.matnr
WHERE c.sap_material_code IS NOT NULL AND m.matnr IS NULL;

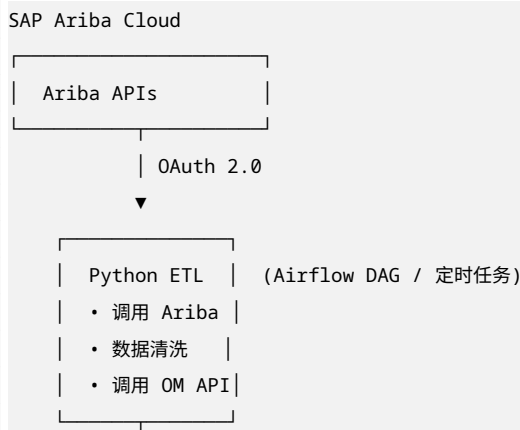
-- 5. 供应商评级缺失检测
SELECT supplier_id, supplier_name
FROM supplier_master
WHERE overall_rating IS NULL AND status = 'Active';

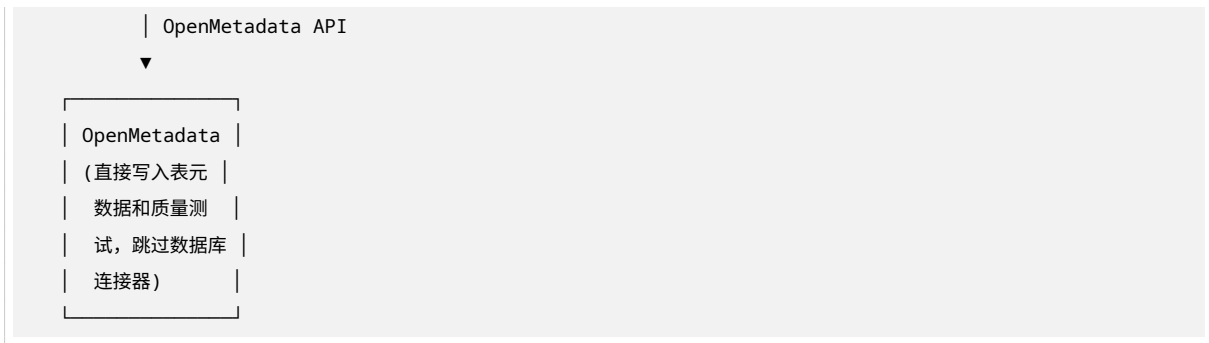
```

方案二：Python 脚本直接调用 OpenMetadata API

适用于不想维护中间数据库的场景，直接用 Python 调用 Ariba API 获取数据，再通过 OpenMetadata Python SDK 写入。

架构





实施代码

```
#!/usr/bin/env python3
"""
SAP Ariba -> OpenMetadata 直连方案
通过 Python SDK 直接将 Ariba 元数据写入 OpenMetadata
"""

import requests
import json
from metadata.ingestion.ometa.ometa_api import OpenMetadata
from metadata.generated.schema.entity.data.table import (
    Table, Column, DataType, TableType
)
from metadata.generated.schema.tests.testCase import TestCase
from metadata.generated.schema.tests.basic import (
    TestCaseResult, TestCaseStatus
)
from datetime import datetime

# ===== 配置 =====
ARIBA_BASE_URL = "https://openapi.ariba.com"
ARIBA_API_KEY = "YOUR_API_KEY"
ARIBA_CLIENT_ID = "YOUR_CLIENT_ID"
ARIBA_CLIENT_SECRET = "YOUR_CLIENT_SECRET"
ARIBA_REALM = "company-prod"

OM_SERVER = "http://localhost:8585/api"
OM_TOKEN = "YOUR_OM_JWT_TOKEN"

# ===== Ariba API 客户端 =====
class AribaAPIClient:
    """SAP Ariba API 客户端"""

    def __init__(self):
        self.token = self._get_token()

    def _get_token(self):
        response = requests.post(
            "https://api.ariba.com/v2/oauth/token",
            data={
```

```

        "grant_type": "client_credentials",
        "client_id": ARIBA_CLIENT_ID,
        "client_secret": ARIBA_CLIENT_SECRET
    }
)
return response.json()["access_token"]

def get_suppliers(self, page=1, page_size=500):
    """ 获取供应商列表 (Supplier Data API) """
    url = f"{ARIBA_BASE_URL}/smv1/vendors/"
    response = requests.post(
        url,
        headers={
            "Authorization": f"Bearer {self.token}",
            "x-api-key": ARIBA_API_KEY,
            "Content-Type": "application/json"
        },
        json={
            "pageSize": page_size,
            "pageToken": None
        }
    )
    return response.json()

# ===== OpenMetadata 写入器 =====
class OpenMetadataWriter:
    """ 将 Ariba 数据写入 OpenMetadata """

    def __init__(self):
        self.om = OpenMetadata(host_port=OM_SERVER, token=OM_TOKEN)

    def create_ariba_service(self):
        """ 创建 Ariba API Service (如果不存在) """
        from metadata.generated.schema.entity.services.apiService import (
            ApiService, ApiConnection, ApiType
        )

        service = ApiService(
            name="SAP-Ariba-Cloud",
            serviceType="REST",
            connection=ApiConnection(
                config={
                    "type": "Rest",
                    "openAPISchemaURL": "https://openapi.ariba.com/api/schema",
                    "token": ARIBA_TOKEN # 加密存储
                }
            )
        )
        return self.om.create_or_update(service)

    def register_supplier_table(self, suppliers_data):

```

```

"""
将 Ariba 供应商数据注册为 OpenMetadata 的 Table 实体

注意: 这里不是创建物理表, 而是创建"逻辑表"来管理 Ariba API 数据
"""

columns = [
    Column(
        name="SupplierId",
        dataType=DataType.VARCHAR,
        dataLength=50,
        description="供应商唯一标识"
    ),
    Column(
        name="SupplierName",
        dataType=DataType.VARCHAR,
        dataLength=200,
        description="供应商名称"
    ),
    Column(
        name="Status",
        dataType=DataType.VARCHAR,
        dataLength=20,
        description="供应商状态"
    ),
    Column(
        name="Country",
        dataType=DataType.VARCHAR,
        dataLength=50,
        description="国家/地区"
    ),
    Column(
        name="OverallRating",
        dataType=DataType.DECIMAL,
        description="综合评级"
    ),
    Column(
        name="LastModifiedDate",
        dataType=DataType.TIMESTAMP,
        description="最后修改时间"
    )
]

table = Table(
    name="Ariba_SupplierMaster",
    displayName="Ariba 供应商主数据",
    description="来自 SAP Ariba Supplier Lifecycle 的供应商主数据",
    tableType=TableType.External,
    columns=columns,
    service=EntityReference(id=service_id, type="apiService")
)

```

```

        created = self.om.create_or_update(table)
        return created

def add_data_quality_tests(self, table_fqn):
    """ 为 Ariba 数据添加质量测试 """
    tests = [
        {
            "name": "Ariba_SupplierId_NotNull",
            "entityFQN": f"{table_fqn}.SupplierId",
            "testDefinition": "columnValuesToBeNotNull"
        },
        {
            "name": "Ariba_SupplierStatus_ValidSet",
            "entityFQN": f"{table_fqn}.Status",
            "testDefinition": "columnValuesToBeInSet",
            "parameterValues": [
                {"name": "allowedValues",
                 "value": "['Active','Inactive','Blocked','UnderReview']"}
            ]
        }
    ]

    for test in tests:
        self.om.create_or_update(TestCase(**test))

# ===== 主流程 =====
def sync_ariba_to_om():
    """ 同步 Ariba 数据到 OpenMetadata """

    # 1. 初始化客户端
    ariba = AribaAPIClient()
    om_writer = OpenMetadataWriter()

    # 2. 获取 Ariba 供应商数据
    print("Fetching suppliers from Ariba...")
    suppliers = ariba.get_suppliers()
    print(f"Retrieved {len(suppliers)} suppliers")

    # 3. 创建/更新 OpenMetadata Service
    service = om_writer.create_ariba_service()

    # 4. 注册供应商表
    table = om_writer.register_supplier_table(suppliers)

    # 5. 添加质量测试
    om_writer.add_data_quality_tests(table.fullyQualified_name)

    # 6. 添加标签和 Glossary
    om_writer.om.patch(
        Table,
        table.id,

```

```
[
    {
        "op": "add",
        "path": "/tags",
        "value": [
            {"tagFQN": "供应商. 主数据", "source": "Glossary"},
            {"tagFQN": "P0", "source": "Classification"}
        ]
    }
]

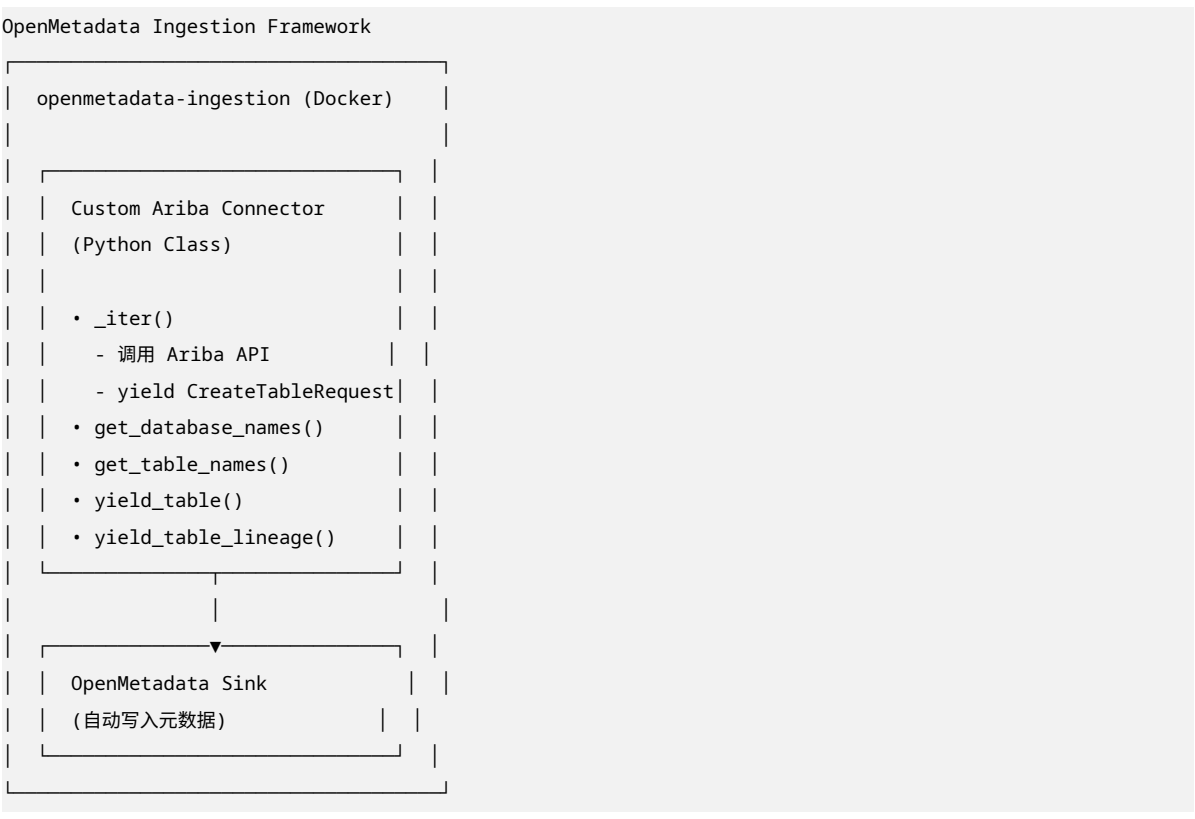
print("$${\vcenter{{\hbox{{\includegraphics[height=\ht\strutbox]{{/usr/share/docminer/emoji/
↪ 2713.png}}}}}$$ Sync completed")

if __name__ == "__main__":
    sync_ariba_to_om()
```

方案三：OpenMetadata Custom Connector（长期方案）

如果需要持续、自动化的 Ariba 数据采集，可以开发一个 OpenMetadata Custom Connector。

架构



Custom Connector 代码

```
#!/usr/bin/env python3
"""
OpenMetadata Custom Connector for SAP Ariba
完整实现示例
"""

import traceback
from typing import Iterable, Optional

from metadata.generated.schema.api.data.createTable import CreateTableRequest
from metadata.generated.schema.entity.data.table import (
    Column, DataType, TableType, Constraint
)
from metadata.generated.schema.entity.services.connections.api.restConnection import (
    RestConnection
)
from metadata.generated.schema.metadataIngestion.workflow import (
    Source as WorkflowSource
)
from metadata.ingestion.api.models import Either, StackTraceError
from metadata.ingestion.api.steps import Source
from metadata.ingestion.ometa.ometa_api import OpenMetadata

import requests
import json

class AribaSource(Source):
    """
    SAP Ariba Custom Connector

    配置方式:
    1. 在 OpenMetadata UI 中创建 Custom Service
    2. Source Python Class Name: connector.ariba_connector.AribaSource
    3. Connection Options 中配置 API 认证信息
    """

    def __init__(self, config: WorkflowSource, metadata: OpenMetadata):
        super().__init__()
        self.config = config
        self.metadata = metadata

        # 从连接配置中读取 Ariba 认证信息
        conn_opts = config.serviceConnection.root.config.connectionOptions or {}
        self.base_url = conn_opts.get("baseUrl", "https://openapi.ariba.com")
        self.api_key = conn_opts.get("apiKey")
        self.client_id = conn_opts.get("clientId")
        self.client_secret = conn_opts.get("clientSecret")
        self.realm = conn_opts.get("realm")
```

```

# 获取 Token
self.token = self._get_oauth_token()

def _get_oauth_token(self) -> str:
    """ 获取 OAuth Access Token """
    response = requests.post(
        "https://api.ariba.com/v2/oauth/token",
        data={
            "grant_type": "client_credentials",
            "client_id": self.client_id,
            "client_secret": self.client_secret
        }
    )
    response.raise_for_status()
    return response.json()["access_token"]

def _ariba_api_call(self, endpoint: str, method="GET", payload=None):
    """ 通用 Ariba API 调用 """
    url = f"{self.base_url}{endpoint}"
    headers = {
        "Authorization": f"Bearer {self.token}",
        "x-api-key": self.api_key,
        "Content-Type": "application/json"
    }

    if method == "GET":
        response = requests.get(url, headers=headers)
    else:
        response = requests.post(url, headers=headers, json=payload)

    response.raise_for_status()
    return response.json()

def get_database_names(self) -> Iterable[str]:
    """ 返回 Ariba 的逻辑数据库名 """
    yield "Ariba_Procurement"
    yield "Ariba_Supplier"

def get_table_names(self, database_name: str) -> Iterable[str]:
    """ 返回每个数据库中的表名 """
    tables_map = {
        "Ariba_Procurement": [
            "PurchaseOrderFact",
            "InvoiceLineItemFact",
            "ContractWorkspaceFact",
            "ProcurementCatalog"
        ],
        "Ariba_Supplier": [
            "SupplierMaster",
            "SupplierPerformance",

```

```

        "SupplierRisk",
        "SupplierQualification"
    ]
}
yield from tables_map.get(database_name, [])

def yield_table(self, table_name: str, schema_name: str) -> Iterable[Either]:
    """
    生成 Table 元数据

    注意: Ariba 是 API 系统, 没有物理数据库的列定义。
    这里需要根据 Ariba API 的响应结构定义列。
    """
    try:
        # 供应商主数据表结构
        if table_name == "SupplierMaster":
            columns = [
                Column(
                    name="InternalId",
                    dataType=DataType.VARCHAR,
                    dataLength=50,
                    description="Ariba 内部供应商 ID",
                    constraint=Constraint.NOT_NULL
                ),
                Column(
                    name="ERPVendorID",
                    dataType=DataType.VARCHAR,
                    dataLength=50,
                    description="ERP 供应商编码 (与 SAP 对应)"
                ),
                Column(
                    name="SupplierName",
                    dataType=DataType.VARCHAR,
                    dataLength=200,
                    description="供应商名称",
                    constraint=Constraint.NOT_NULL
                ),
                Column(
                    name="LegalName",
                    dataType=DataType.VARCHAR,
                    dataLength=200,
                    description="供应商法定名称"
                ),
                Column(
                    name="Status",
                    dataType=DataType.VARCHAR,
                    dataLength=20,
                    description="状态: Active/Inactive/Blocked/UnderReview"
                ),
                Column(
                    name="CountryCode",

```



```

        dataType=DataType.VARCHAR,
        dataLength=10,
        description="国家代码"
    ),
    Column(
        name="OverallRating",
        dataType=DataType.DECIMAL,
        description="综合评级 0-100"
    ),
    Column(
        name="RiskScore",
        dataType=DataType.DECIMAL,
        description="风险评分 0-100"
    ),
    Column(
        name="LastModified",
        dataType=DataType.TIMESTAMP,
        description="最后修改时间"
    )
]

# 采购订单事实表
elif table_name == "PurchaseOrderFact":
    columns = [
        Column(name="POId", dataType=DataType.VARCHAR, dataLength=50),
        Column(name="PONumber", dataType=DataType.VARCHAR, dataLength=50),
        Column(name="SupplierId", dataType=DataType.VARCHAR, dataLength=50),
        Column(name="SupplierName", dataType=DataType.VARCHAR, dataLength=200),
        Column(name="MaterialCode", dataType=DataType.VARCHAR, dataLength=50,
            description="物料编码 (可能与 SAP 编码不同) "),
        Column(name="MaterialDescription", dataType=DataType.VARCHAR, dataLength=500),
        Column(name="Quantity", dataType=DataType.DECIMAL),
        Column(name="Unit", dataType=DataType.VARCHAR, dataLength=10),
        Column(name="UnitPrice", dataType=DataType.DECIMAL),
        Column(name="Currency", dataType=DataType.VARCHAR, dataLength=5),
        Column(name="TotalAmount", dataType=DataType.DECIMAL),
        Column(name="PODate", dataType=DataType.DATE),
        Column(name="NeedByDate", dataType=DataType.DATE),
        Column(name="Status", dataType=DataType.VARCHAR, dataLength=20),
        Column(name="Plant", dataType=DataType.VARCHAR, dataLength=10,
            description="工厂代码"),
        Column(name="CostCenter", dataType=DataType.VARCHAR, dataLength=20)
    ]

else:
    columns = []

# 生成 CreateTableRequest
table_request = CreateTableRequest(
    name=table_name,
    displayName=f"Ariba {table_name}",

```

```

        description=f"SAP Ariba {table_name} data from API",
        tableType=TableType.External,
        columns=columns,
        databaseSchema=f"{self.config.serviceName}.{schema_name}"
    )

    yield Either(right=table_request)

except Exception as exc:
    yield Either(
        left=StackTraceError(
            name=f"Error yielding table {table_name}",
            error=str(exc),
            stackTrace=traceback.format_exc()
        )
    )

def yield_table_lineage(self, table_name: str) -> Iterable[Either]:
    """
    生成血缘关系

    例如: Ariba PurchaseOrderFact.SupplierId -> SAP Supplier Master
    """
    # 血缘关系在数据实际集成后定义
    pass

def _iter(self) -> Iterable[Either]:
    """
    核心迭代方法
    OpenMetadata Ingestion Framework 会遍历此方法
    """
    for database_name in self.get_database_names():
        for table_name in self.get_table_names(database_name):
            yield from self.yield_table(table_name, database_name)

# ===== 注册 Connector =====
# 在 setup.py 中注册
"""
from setuptools import setup

setup(
    name="openmetadata-ariba-connector",
    version="1.0.0",
    packages=["connector"],
    install_requires=[
        "openmetadata-ingestion>=1.12.0",
        "requests>=2.28.0"
    ],
    entry_points={
        "openmetadata.source": [

```

```

        "ariba = connector.ariba_connector:AribaSource"
    ]
}
)
"""

```

Custom Connector 部署步骤

```

# Step 1: 构建自定义镜像
cat > Dockerfile << 'EOF'
FROM openmetadata/ingestion:1.12.6

WORKDIR /ingestion
USER airflow

# 安装自定义 Connector
COPY connector/ connector/
COPY setup.py .
RUN pip install --no-deps -e .
EOF

docker build -t openmetadata-ingestion-ariba:latest .

# Step 2: 修改 docker-compose 使用自定义镜像
# docker-compose-postgres.yml
services:
  openmetadata_ingestion:
    image: openmetadata-ingestion-ariba:latest
    # ... 其他配置

# Step 3: 在 OpenMetadata UI 中配置
# 1. Settings > Services > API Services > Add New Service
# 2. 选择 "Custom"
# 3. Source Python Class Name: connector.ariba_connector.AribaSource
# 4. Connection Options:
#    {
#      "baseUrl": "https://openapi.ariba.com",
#      "apiKey": "YOUR_API_KEY",
#      "clientId": "YOUR_CLIENT_ID",
#      "clientSecret": "YOUR_CLIENT_SECRET",
#      "realm": "company-prod"
#    }

# Step 4: 运行元数据采集

```

方案四：CSV 导出 → S3 → OpenMetadata（快速验证）

适用于 **POC 阶段**快速验证，不需要开发。

步骤

Step 1: Ariba 中导出 CSV

Ariba UI 操作:

1. 登录 Ariba
2. 导航到 Reports > Analytical Reporting
3. 选择报表模板:
 - "Supplier Dimension" (供应商维度)
 - "Purchase Order Fact" (采购订单事实)
4. 点击 Actions > Configure Export > CSV
5. 点击 Export Data, 下载 ZIP 文件
6. 解压后得到 CSV 文件

Step 2: 上传 CSV 到 S3/本地

```
# 创建 S3 Bucket
aws s3 mb s3://company-ariba-landing-zone

# 上传 CSV
aws s3 cp supplier_dimension.csv s3://company-ariba-landing-zone/supplier/2026/05/01/
aws s3 cp purchase_order_fact.csv s3://company-ariba-landing-zone/po/2026/05/01/
```

Step 3: OpenMetadata S3 Connector 配置

```
{
  "serviceName": "Ariba-S3-Landing",
  "serviceType": "S3",
  "connection": {
    "config": {
      "type": "S3",
      "awsConfig": {
        "awsAccessKeyId": "YOUR_ACCESS_KEY",
        "awsSecretAccessKey": "YOUR_SECRET_KEY",
        "awsRegion": "cn-north-1"
      },
    },
    "bucketName": "company-ariba-landing-zone",
    "prefix": "supplier/",
    "generateSampleData": true
  }
}
```

局限性:

- 需要手动导出, 无法自动同步
- OpenMetadata S3 Connector 对 CSV 的结构化解析能力有限
- 不适合生产环境

各方案对比与选型建议

维度	方案一（HANA/DB）	方案二（Python API）	方案三（Custom Connector）	方案四（CSV/S3）
开发工作量	中（需 ETL 开发）	中（Python 脚本）	高（Connector 开发）	低
维护成本	低（数据库稳定）	中（需维护脚本）	高（需随 OM 升级维护）	高（手动操作）
数据时效性	小时级	小时级	小时级	日级/手动
可扩展性	高	中	高	低
血缘追踪	完整（DB 级）	需手动定义	可编程定义	无
质量测试	SQL 测试（完整）	需 SDK 调用	原生支持	有限
需要组件	数据库 + ETL	Python 环境	Docker + Python	S3
推荐阶段	生产环境	过渡期/临时	长期规划	POC

选型决策树

是否有 SAP HANA / 数据库集成经验？

是 → 方案一（HANA/DB）

推荐

理由：最稳定，血缘追踪完整，运维简单

否 → 需要多快上线？

1-2 周 → 方案二（Python API）

理由：开发快，灵活性高

2-4 周 → 方案四（CSV） → 快速验证
然后迁移到方案一或二

2-3 月 → 方案三（Custom Connector）

理由：最原生，长期价值最高

适合有专职 OM 运维团队的企业

制造业特殊考虑：Ariba 物料数据与 SAP 物料数据的关联

在离散制造业中，SAP Ariba 和 SAP ERP 的物料数据往往存在以下关联场景：

场景 1：供应商物料编码 SAP 物料编码映射

SAP Ariba		SAP ERP
CatalogItemID: CAT-001		Material: 10000001
SupplierPartNum: S-100	→	MATNR: 10000001
Supplier: SUP-001	映射	Description: 螺栓 M8
UnitPrice: 0.50 USD		BaseUnit: PCS

Kimi.ai 生成

血缘追踪方案:

- 在 OpenMetadata 中手动创建血缘: Ariba.ProcurementCatalog.SupplierPartNum → SAP.MARA.MATNR

场景 2: 供应商绩效 → 采购决策

Ariba Supplier Performance

SupplierId: SUP-001	
OnTimeDeliveryRate: 95%	→ OpenMetadata 质量测试
QualityScore: 88	测试规则:
PriceCompetitiveness: 92	- OnTimeDeliveryRate > 90%
OverallRating: 91.7	- QualityScore > 80

场景 3: 跨系统一致性校验

```
-- OpenMetadata Test Library: Ariba-SAP 跨系统一致性测试

-- 测试 1: Ariba 中的供应商必须在 SAP 中存在
SELECT a.supplier_id, a.supplier_name
FROM ariba_dw.supplier_master a
LEFT JOIN sap_erp.lfa1 s ON a.erp_vendor_id = s.lifnr
WHERE s.lifnr IS NULL AND a.status = 'Active';

-- 测试 2: Ariba 采购价格与 SAP 信息记录价格差异
SELECT
    a.material_code,
    a.unit_price as ariba_price,
    i.netpr as sap_price,
    ABS(a.unit_price - i.netpr) / i.netpr * 100 as diff_percent
FROM ariba_dw.purchase_order_fact a
JOIN sap_erp.eina i ON a.material_code = i.matnr
WHERE ABS(a.unit_price - i.netpr) / i.netpr * 100 > 10; -- 差异 > 10%

-- 测试 3: Ariba 计量单位与 SAP 不一致
SELECT
    a.material_code,
    a.unit as ariba_unit,
    m.meins as sap_unit
FROM ariba_dw.procurement_catalog a
JOIN sap_erp.mara m ON a.sap_material_code = m.matnr
WHERE a.unit != m.meins;
```

附录: Ariba API 关键端点速查

API	端点	用途	制造业场景
Supplier Data Analytical Reporting (Async)	/smv1/vendors/	供应商数据分页查询	供应商主数据同步
	/api/reporting-job/v1/{realm}/jobs	异步报表导出	大批量历史数据
Analytical Reporting (Sync)	/api/analytics-reporting-view/v1/{realm}/views	同步报表查询	小批量实时查询
Sourcing Reporting	/sourcing-reporting-details/v1/{realm}/views	寻源报表	物料寻源分析
Procurement	/procurement/v2/{realm}/...	采购操作	订单/合同数据

常用报表视图模板 (View Templates)

视图模板	数据内容
SupplierDimension	供应商维度表
SupplierSpentFact	供应商支出事实
ItemMasterFact	物料主数据事实
PurchaseOrderFact	采购订单事实
InvoiceLineItemFact	发票行项目事实
ContractWorkspaceFact	合同工作区事实

文档版本: v1.0 最后更新: 2026-05-02 适用 OpenMetadata 版本: 1.12.x